

SYNOPSIS

Blog Web Application with User Authentication and AI-Assisted Content Management

A Flask-based dynamic blogging platform for publishing articles, managing readers, and improving editorial workflow

Project Type	Web Application / Academic Project Synopsis
Technology Stack	Python, Flask, Flask-SQLAlchemy, SQLite, Jinja2, Bootstrap, Flask-Login, Flask-WTF, CKEditor, Gmail SMTP, Gemini API
Prepared For	Supervisor / Project Guide
Prepared By	Student Name: _____
Date	May 2026

1. Introduction

The proposed project is a dynamic blog web application designed to allow users to read articles, register securely, participate through comments, and interact with a structured publishing system. The application is developed using the Flask web framework and follows a server-rendered architecture with HTML templates, Bootstrap styling, SQLite database storage, and form-based user interaction.

Unlike a static blog, this system includes user accounts, authentication, email-based OTP verification, password reset, comments, admin-only article management, and an AI-assisted writing workflow. The project therefore demonstrates both core web development concepts and practical features expected in a modern content management application.

2. Problem Statement

Many basic blog websites only display fixed content and do not provide secure user participation or convenient article management. Administrators often need a simple interface to create and edit posts, while readers need a reliable way to create accounts, log in, comment, and recover access if they forget their password. The project addresses this problem by building a complete blogging platform with authentication, database-backed content, and protected administrative controls.

3. Objectives

- To develop a responsive web-based blog platform using Flask and Bootstrap.
- To provide secure user registration, login, logout, and session management.
- To implement email OTP verification for new account registration and password reset.
- To allow authenticated users to comment on blog posts.
- To allow only the administrator to create, edit, delete, and monitor blog content.
- To integrate AI support for generating article drafts, summaries, titles, and tags.
- To maintain persistent data using SQLite and SQLAlchemy ORM models.

4. Scope of the Project

The scope of the system covers the development of a functional blog application where readers can browse articles and registered users can participate in discussions. The administrator can manage blog posts from protected routes and use AI-assisted tools to reduce the effort of preparing titles, summaries, tags, and full drafts. The application is suitable as a learning project, prototype content management system, or base for a small publishing platform.

The current implementation focuses on core functionality rather than large-scale deployment. The system uses SQLite for local persistence and Gmail SMTP for OTP delivery. It also includes fallback logic for AI content generation so the application can continue to operate even when an external AI API key is unavailable or a request fails.

5. Existing System and Proposed System

Aspect	Existing / Basic Blog	Proposed Application
Content	Static or manually updated pages	Database-backed posts with create, edit, delete, summary, and tags
Users	No secure user workflow	Registration, login, logout, account verification, and password reset
Security	Limited access control	Admin-only dashboard and protected content management routes
Interaction	Reader participation may be absent	Authenticated comments on individual posts
Editorial Support	Manual writing only	Gemini-assisted draft, title, summary, and tag generation with fallback logic

6. Methodology

1. Requirement analysis was performed by identifying the main users: visitors, registered users, and administrator.
2. The application structure was implemented using Flask routes, Jinja2 templates, WTForms, and SQLAlchemy models.
3. Database models were created for users, blog posts, and comments, with relationships between authors, posts, and discussions.
4. Authentication was implemented using Flask-Login with password hashing and account verification through email OTP.
5. Administrative operations were protected using a custom admin-only decorator that restricts access to the user with administrator privileges.
6. AI-assisted editorial features were integrated using the Gemini API, while fallback functions were added for reliability.

7. Hardware and Software Requirements

Hardware	A standard desktop/laptop with 4 GB RAM or higher and sufficient local storage for Python environment and SQLite database.
Operating System	Windows, Linux, or macOS capable of running Python and Flask.
Frontend	HTML, CSS, Bootstrap, Jinja templates, CKEditor for rich-text editing.
Backend	Python with Flask, Flask-Login, Flask-WTF, Flask-SQLAlchemy, Flask-Bootstrap, Flask-CKEditor.
Database	SQLite database file named blog.db.
External Services	Gmail SMTP for OTP emails and Gemini API for AI-assisted content generation.

8. System Modules

User Registration and Verification: Users register with name, email, and password. A six-digit OTP is sent to the registered email address. The account becomes active only after successful OTP verification.

Login and Session Management: Verified users can log in securely. Flask-Login manages authenticated sessions and controls access to protected actions.

Password Reset: Users can request a password reset OTP by email. After entering the correct OTP and matching new passwords, the password is updated using a secure hash.

Blog Post Management: The administrator can add, edit, and delete blog posts. Each post contains title, subtitle, image URL, body, publication date, summary, tags, and author information.

Comment System: Authenticated users can post comments on individual blog articles. Comments are linked to both the user and the parent blog post.

Admin Dashboard: The administrator dashboard displays summary statistics such as total posts, users, comments, tagged posts, recent posts, recent comments, and newest users.

AI-Assisted Content Generation: The system can generate article drafts from a topic and can also generate improved titles, summaries, and tags from existing article content using the Gemini API.

9. Database Design

Table / Model	Important Fields	Purpose
User	id, name, email, password, is_verified, registration OTP fields, reset OTP fields	Stores account data, verification state, and OTP details.
BlogPost	id, author_id, title, subtitle, date, body, img_url, summary, tags	Stores published article information and AI-generated metadata.
Comments	id, text, author_id, parent_post_id	Stores reader comments linked to users and blog posts.

10. Data Flow

- A visitor opens the home page and views all available posts in descending order.
- A new user registers and receives a verification OTP through email.
- After verification, the user can log in and comment on posts.
- The administrator logs in and accesses the dashboard and protected post management pages.
- When creating or editing a post, the administrator may request AI-generated drafts, summaries, titles, and tags.
- All accepted post, user, and comment data is stored in the SQLite database through SQLAlchemy.

11. Security Features

- Passwords are stored using Werkzeug password hashing rather than plain text for new and reset passwords.
- Registration and password reset OTPs expire after a fixed time period and include resend cooldown control.
- Administrative routes are protected by login checks and an admin-only authorization decorator.
- Unverified accounts are prevented from logging in until the email verification process is completed.
- Environment variables are used for secret keys, mail credentials, and external AI API keys.

12. Key Features of the Application

- Responsive user interface built with Bootstrap and reusable templates.
- Rich-text blog editing through CKEditor.
- Secure registration, login, logout, verification, and password recovery workflows.
- Email delivery of OTPs through Gmail SMTP.
- Post summaries and tags for improved readability and organization.
- Admin dashboard for monitoring users, posts, comments, and recent activity.
- AI-assisted editorial workflow with graceful fallback when the external service is unavailable.
- Database schema migration checks to add missing columns when the application starts.

13. Expected Outcome

The expected outcome of the project is a working Flask-based blog application that demonstrates a complete content publishing workflow. The system enables visitors to read posts, users to register and comment, and the administrator to manage published content. The OTP-based verification and password reset features improve account reliability, while the AI-assisted content generation features make the project more current and useful for editorial work.

14. Limitations

- The application currently uses SQLite, which is suitable for development and small deployments but may need replacement with PostgreSQL or MySQL for heavy production use.
- The administrator role is identified by a fixed user id, which can be improved through a role-based access control table.
- Email delivery depends on correct SMTP credentials and network availability.
- AI output depends on the configured Gemini API key and external service availability.

15. Future Enhancements

- Add role-based permissions for multiple administrators and editors.
- Add post categories, search, pagination, likes, bookmarks, and reader profiles.
- Deploy the application on a cloud platform with a production database.
- Add image upload support instead of relying only on external image URLs.
- Improve analytics with views, trending posts, and comment moderation tools.
- Add automated tests for authentication, OTP flows, post management, and comments.

16. Conclusion

This project presents a practical implementation of a secure and interactive blogging platform using Python Flask. It combines fundamental web application concepts such as routing, templates, forms, sessions, database models, and access control with modern additions such as OTP-based account verification and AI-assisted content preparation. The system is suitable for demonstrating full-stack web development skills and can be extended into a more advanced content management platform in future work.